

Project: Yaai's Journal - a two-dimensional adventure game that teaches Python programming and Southern Thai culture

by

Master Krittidet Koedsiri

Master Pasit Anusonadisai

Master Chewin Klingbuathong

Grade 9 (Mattayom 3), Benjamarachutit School

Under the Secondary Educational Service Area Office Nakhon Si Thammarat

This report is a component of the computer project, software category,
for the 74th Student Arts and Crafts Competition, Academic Year 2569

Level ..... Area ..... Region .....

3 December 2569

A

Project name	Yaai's Journal - a two-dimensional adventure game that teaches Python programming and Southern Thai culture
Report authors	1. Master Krittidet Koetsiri 2. Master Pasit Anusonthi-adisai 3. Master Chiewin Klingbuathong
Project advisers	1. Mrs. Bharati Sattangmongkol 2. Mrs. Niparat Niamsong
Affiliation	Secondary Educational Service Area Office Nakhon Si Thammarat; Benjamarachutit School; Grade 9 (Mattayom 3)

Abstract

Purpose. The objective of this project is to develop a computer-based teaching aid that teaches the fundamentals of the Python programming language and Southern Thai language and culture, using game-based learning. Method. We designed and built an original 2-D educational adventure game, Yaai's Journal, in Python with pygame-ce, using a data-driven story engine (24 hand-written stops in five acts), an automatic puzzle engine with eight puzzle archetypes that execute and validate the player's typed Python code inside a restricted sandbox, a generative pixel-art system, per-region weather, two film-style cutscenes, and a test-driven development process with 201 automated test cases (3,283 test lines) that run headless. Results. The finished program contains 17,762 lines of game code in 52 modules plus a 3,283-line automated test suite (75 Python files, 21,396 lines in total), and the world carries more than 21,000 words of story, dialogue and lore: 24 stops, 40 heritage landmarks, 25 side quests, 18 lore documents and 74 road encounters. All 24 puzzle solutions were run through the game's own validator and accepted, while every pre-filled starting code was run through it and rejected, so no lesson can be skipped. The full test suite and the complete game run on a normal classroom laptop without installation beyond two pip packages. Conclusion. A story-driven quest structure in which the story can only advance by writing real Python is a practical, testable and repeatable form of teaching aid for the computing classroom, and it can be extended to other provinces and other subjects by editing data files only.

Keywords: Python; educational game; game-based learning; Southern Thailand; test-driven development; pygame-ce; vocabulary; local culture

Table of Contents

Chapter 1 Introduction....6
1.1 Background and Significance of the Project....6
1.2 Objectives of the Project....6
1.3 Hypothesis of the Project....7
1.4 Scope of the Project....7
1.5 Expected Benefits of the Project....7
1.6 Definitions....8
Chapter 2 Review of Related Literature....9
2.1 Learning by building something meaningful....9
2.2 Enjoyment and immediate feedback....9
2.3 Typed code versus blocks....9
2.4 Python in the curriculum....9
2.5 pygame-ce and code-generated graphics....9
2.6 Security of player-submitted code....10
2.7 Test-driven development....10
2.8 Place-based and cultural education....10
2.9 Related research and resources....10
Chapter 3 Equipment and Research Methodology....12
3.1 Problem definition....12
3.2 Study of theory and tools....12
3.3 System design....12
3.4 Development....13
3.5 Testing....14
3.6 Publication and installation....14
3.7 Equipment....14
Chapter 4 Results and Discussion....16
4.1 What was built....16
4.2 The teaching ladder: 24 puzzles, 8 archetypes....16
4.3 Correctness evidence from the test suite....17
4.4 Screenshots of the delivered program....17
4.5 Discussion....20
Chapter 5 Conclusion and Discussion....22

5.1 Conclusions....22
5.2 Discussion of the results....22
5.3 Problems encountered and how they were solved....23
5.4 Suggestions and future development....23
References....24
Appendix A The 24 stops, their Thai words and their puzzles....25
Appendix B Three example tablets and their solutions....27
Appendix C How to run, the project tree, the test suite and the classroom quiz....28

List of Tables and Figures

Table 3.1 The conceptual framework of the project (Input - Process - Output)

Table 3.2 Example test files and what they check

Table 3.3 Equipment and software used in the project

Table 4.1 Main modules of the delivered game

Table 4.2 The eight puzzle archetypes and the Python concepts they teach

Table 5.1 Objective-to-evidence summary

Figure 4.1 The cover screen: title, generated pixel art and the menu (capture of the delivered build, 1920 x 1080)

Figure 4.2 Walking the coast: the overworld with HUD, quest prompt and the stop beacon

Figure 4.3 Typewriter dialogue: Noi teaches one beat of the story at a time

Figure 4.4 A rune tablet: the puzzle status rule and the prefilled code, ready to edit

Figure 4.5 The journal, opened from the overworld: collected pages, words and cards

Figure 4.6 The sailing map: the 24 stops in five acts across the Southern coast

Chapter 1 Introduction

1.1 Background and Significance of the Project

In the computing classroom, Python is recommended by the national computing curriculum as a first real programming language, and most of our class had already seen print() and variables in grade 7. We wanted more practice time and a stronger reason to write code, and we wanted a tool that works offline and connects to our own lives.

Our school is in Nakhon Si Thammarat, in the South of Thailand, and the region is the one we live in. We wanted the game to travel through that coast, from Chumphon to Narathiwat, with its provinces, islands, food, wise grandmothers, the old navy of the Prince of Chumphon, turtle conservation, the floating school, the lanterns that remember the 2004 Indian Ocean tsunami, and the Wai Khru ceremony that closes the story. Choosing our own region keeps the cultural content close to the students who will play it, and it makes the game something our class can recognise from the first screen. Combining the two problems gave us one idea: a journal that cannot be read unless the player programs.

The result is Yaai's Journal. The player is a young apprentice who travels with the puppet teacher Noi around the Southern coast, restoring pages from Yaai, the legendary teacher of the sea, torn from her journal by the storm. Each of the 24 stops teaches one Thai word and one piece of local history; each page of the journal is won at a rune tablet by writing a small, real Python program; the final tablet is a gauntlet that combines everything learned on the journey. Between the stops the road is alive with 74 small encounters, 25 side quests and 40 heritage landmarks, so the culture arrives while walking, not only in dialogue.

We chose real typed code instead of ready-made drag-and-drop blocks because Python is the language the curriculum and the examinations actually use. To keep beginners from stopping halfway, the game has three layers of help. The first layer is the puzzle rule written on the tablet with an example of the required output. The second layer is the prefilled starter code, which never passes the validator, so editing is easier than starting from a blank page. The third layer is the hint and Noi's dialogue, which explain the idea one step at a time; when the player writes wrong code, the system does not show a program-style error message, but says which rule has not passed yet and what to fix first.

1.2 Objectives of the Project

- To develop a free computer-based teaching aid that teaches Python fundamentals through typed, executed and automatically validated code, not through passive reading.
- To integrate Python teaching with Thai language learning: 24 Southern Thai words (with romanisation and meaning) are taught inside the same quests that teach code.
- To preserve and share the cultural knowledge of Southern Thailand (people, places and events) in a medium our generation actually uses.

- To build the teaching aid to an engineering standard: data-driven content, a restricted code-execution sandbox, and a serious automated test suite.
- To document the creative process so that other students can reuse the design for their own provinces and subjects.

1.3 Hypothesis of the Project

Hypothesis 1: if the starter code on every rune tablet is written so that it fails the tablet's own validator, the player cannot read the next part of the story until the code is corrected, and the 24 solutions we prepared are accepted by the same validator.

Hypothesis 2: if the story content and the game code are kept apart, a new stop can be added by editing one data file, and the existing tests catch missing or malformed data before it is used.

1.4 Scope of the Project

What the project does

- A single-player, offline, keyboard-first game for Windows (Linux/macOS supported by pygame-ce).
- Five acts and 24 story stops from Chumphon to Narathiwat, each with dialogue, a local host, one Thai word, culture cards, a local activity and one Python puzzle; 40 heritage landmarks, 25 side quests and 74 road encounters wait between the stops.
- 24 programming lessons covering: print, variables, ==, if/else, for and while loops, lists, dictionaries, f-strings, functions with try/except, and small classes.
- Extra systems: two film-style cutscenes, save/load, key rebinding, options, mouse support, an eight-tab journal with a quest log, a world map, a village-naming scene, a fishing mini-system, per-region weather, and a fully code-generated pixel-art look.
- A 201-case automated test suite that runs without a display.

What the project does not do

- Multiplayer, mobile/touch versions, voice acting, an online scoreboard, and 3-D graphics.

1.5 Expected Benefits of the Project

- Students get a programming practice medium in which they type real code and see the result immediately, which lowers the fear of starting to write.
- Teachers get a teaching medium that installs easily, works without an Internet connection and checks answers automatically, leaving more time for students who are stuck.
- Southern Thai cultural content is conveyed in a form young people actually use, compiled systematically from family accounts and local documents.

- The school and future students can add stops for other provinces by editing a data file, without touching the game's code.

1.6 Definitions

Term	Meaning in this project
Stop	One of the 24 places the player visits; the unit of story, culture and code.
Rune tablet	The stone that asks for a Python program; passing it wins the page.
Page	The quest reward; 24 stop pages + 2 ceremony pages = 26 journal pages in total.
Spark	A hidden word the host teaches in the wild; it unlocks a locked tablet.
Puzzle archetype (kind)	The validator family of a puzzle: say, lines, set, table, func, safe, class, gauntlet.
Prefill	The code already inside the tablet when the player arrives; it never passes.
Validator	The function that runs the player's code and decides whether it is correct.
Sandbox	The restricted execution environment (safe builtins, captured output, timeout).
Culture card	A collectible summary of the cultural knowledge of a stop.
TDD	Test-driven development: automated tests are part of the development loop.
Landmark	One of the 40 heritage sites (wat, chedi, stupa, mosque, old town) that stands in its real town; walking to it fills a journal page.
Side quest	One of the 25 optional local stories that a host offers; marked ON, READY or DONE in the journal's quest log.
Cutscene	One of the two film-style scenes (the storm that opens the game, the cup that closes it) played by the shot-and-camera engine.

Chapter 2 Review of Related Literature

To design the game and its validator we studied educational theory, research on teaching programming, and the technical documentation of the tools we use. The topics are summarised as follows.

2.1 Learning by building something meaningful

Papert's constructionism, "learning happens especially well when the learner builds a public product", is the pedagogical root of our design (Papert, 1980): the public product here is the growing journal itself. Vygotsky's zone of proximal development (Vygotsky, 1978) explains the puzzle ladder: each tablet sits slightly beyond what the player can already do, and the tablet's hint plus the keeper's dialogue act as the scaffold.

2.2 Enjoyment and immediate feedback

Csikszentmihalyi's flow conditions - clear goals, immediate feedback, and challenge matched to skill (Csikszentmihalyi, 1990) - are met structurally by our validator: the goal and the acceptance rule are written on the tablet, feedback is instant (the tablet prints the result of the run), and the archetype ladder raises the challenge from print() to classes. Prensky (2001) argues that digital games are a natural language for young learners; our project is therefore not a lesson with game elements added, but a lesson built in the form of a whole game.

2.3 Typed code versus blocks

Weintrop and Wilensky (2017) found that beginners found text-based programming more difficult initially, and that hybrid or scaffolded entry points help. Our design takes the scaffolding route: real typed Python, but with a prefilled edit buffer (which always fails), a 12-line primer on the screen, and hints written by the keeper. The first tablet is preceded by a seven-beat tutorial before the player reaches it.

2.4 Python in the curriculum

The Institute for the Promotion of Teaching Science and Technology (IPST, 2017) names Python in the learning standards of the science and technology learning area for lower secondary education. We therefore ordered the lessons along the sequence the curriculum actually teaches - output, variables, conditions, loops, data structures, functions, error handling and classes - so that what players see in the game can be used in the classroom immediately.

2.5 pygame-ce and code-generated graphics

pygame-ce is a Python 2-D game library maintained by the community as a drop-in continuation of pygame, supporting Windows, macOS and Linux (pygame-ce, 2026). The game uses pygame-ce as its only graphics library, together with numpy for the generative art. Every picture in the game is drawn by code in the modules artgen.py and tiles.py, so

the project contains no external image files at all; this keeps it small and lets colours and shapes be changed by editing numbers in code. The audio uses Kevin MacLeod's CC-BY soundtrack, credited in the project's CREDITS.md file.

2.6 Security of player-submitted code

Running code received from the player requires care, because such code can crash the program or reach the machine's files. We studied ways of restricting the execution environment and chose the simplest approach that is sufficient for a classroom: a whitelist of permitted builtins, capturing printed output instead of letting it reach the screen, a limit on code length, and a maximum execution time. We state in the accompanying document that this sandbox was built for the puzzles inside the game; it is not a production-grade security tool.

2.7 Test-driven development

Beck (2003) proposed writing the tests before, or alongside, the features they check, so that the intended behaviour is pinned down from the start and changes can be made with confidence. We used this method for every part of the game, because a teaching aid that younger students will edit needs a safety net that keeps it from failing silently. The test suite is therefore part of the delivered work, not a temporary development tool.

2.8 Place-based and cultural education

Place-based education ties learning to the student's own surroundings (Sobel, 2004). Our 24 stops follow the real geography of Southern Thailand in five acts, from the Gulf (Chumphon to Surat Thani and the western islands) through the Andaman (Ranong to Phuket, Krabi and Trang) to the rain country and the deep South (Satun to Narathiwat, ending at the Wai Khru ceremony). Forty heritage landmarks - the Chaiya chedi, the Phra Mahathat stupa of Nakhon Si Thammarat, the old shophouses of Phuket, the central mosque of Pattani - stand in the towns that own them. Cultural content is kept factual and respectful: the Prince of Chumphon and the Thai navy, turtle conservation on Ko Tao, the matriarchs of Ko Lanta, the ocean-line of the 2004 tsunami remembered at Khao Lak, the shadow-puppet (nang talung) troupe of Phatthalung with its hero/clown/demon voice counts, and the indigo-dyed batik of Yala.

2.9 Related research and resources

- IPST's Python teaching documents are the basis for the order of the content and the choice of exercises.
- The pygame-ce documentation and community examples of 2-D games guided the scene management, the game loop and keyboard event handling.
- Research on teaching programming with games that record the player's progress supports the in-game statistics that let teachers see a student's learning path.

- Articles on learning-game design that insist feedback should follow effort, not correct answers, led us to make the validator answer every mistake with the rule and Noi's explanation.

Chapter 3 Equipment and Research Methodology

The project uses design-and-development research, organised as five phases following ADDIE (Analyse, Design, Develop, Implement, Evaluate). Every phase leaves evidence that can be checked in the delivered program itself.

3.1 Problem definition

Studying how computing is taught in our classrooms showed three problems. First, the time for programming practice is short, and exercises come without a context that shows what the knowledge is for. Second, measuring each student's progress is hard when a teacher has many students. Third, local content is taught through short passages that give students no picture of it. We therefore set three requirements for the system: lessons must be short and finishable in a few minutes; answers must be checked immediately by the system itself; and local content must be tied to the same lesson the student is practising.

3.2 Study of theory and tools

- Programming: Python 3.14 and the handling of beginners' errors, so that on-screen explanations use words students know.
- Graphics and sound: drawing pixel art from code with pygame-ce, and background music with a fallback for machines without an audio device.
- Player experience: designing the play path, rewarding effort, and reducing frustration when the player answers wrong.
- Content: collecting Southern Thai words and stories from family accounts and local documents, then compiling them as game data.

3.3 System design

The project works to an Input - Process - Output framework, shown in Table 3.1.

Table 3.1 The conceptual framework of the project (Input - Process - Output)

INPUT	PROCESS	OUTPUT
MoE computing standardsPython 3.14 + pygame-ceSouthern-Thai heritage research (people, places, words, stories)free open assets	story engine / quest loop; puzzle engine + sandbox; validator + arcade; generative pixel art; per-region weather; multi-mood audio; 201-case test suite	Yaai's Journal game (24 stops in five acts, 26 pages); 3,283-line test suite; teaching report (this document)

3.3.1 Play structure

The game is divided into eleven scenes: the title screen, the cold-open storm cutscene, the prologue at Yaai's house, the village as the home base, the explorable overworld, the close-up scene at a stop, the cutscene player, the village-naming scene, the shrine, the well, and the ending with the epilogue. The puzzle tablet, the journal, the sailing map and the settings open as overlays on top, so the player alternates between exploring, reading and writing code and is never stuck in one screen.

3.3.2 Data structure

All content is stored as dictionaries in data files, clearly separated from the game code: story.py for the 24 stops, landmarks.py for the 40 heritage sites, sidequests.py for the 25 side quests, encounters.py for the 74 road encounters and cutscenes.py for the two cutscenes. Each stop's data holds its name, map position, act, dialogue, activity, Thai word with reading and meaning, culture card and the text of the page awarded. Each puzzle's data holds its name, archetype, starter code, rules that must pass, hints, praise and the values the validator expects. With this structure, adding a new stop means adding one dictionary, not changing the game's code.

3.3.3 Screen design

The game draws at an internal resolution of 640 by 320 pixels and scales it four times with nearest neighbour into a 2560 by 1280 window, shrinking to fit smaller screens. Text uses a pixel font drawn inside the game. The status bar at the top shows the pages collected, the act name (one of five), the next goal and two power values, so players know where they are in the story without opening a menu.

3.4 Development

3.4.1 Story and quest mechanism

We wrote the story mechanism as a state machine of scenes; scene changes come from events inside the game, not from a skip button. The quest system records whether the player has collected all the sparks and unlocks the locked tablets when they are complete. Each page is stored with its chapter name, Thai word and culture card, so the journal displays progress correctly.

3.4.2 Puzzle system and sandbox

The module runner.py takes code from the screen and runs it in a restricted namespace with a whitelist of permitted builtins, captures what the code prints, limits code length to 6,000 characters, and sets a maximum running time for each attempt. The module tablets.py holds the eight validator archetypes; each kind checks a different shape of knowledge - printed output, variable values, the order of lines, or the properties of a class the player builds. Players may correct their code any number of times without penalty, and every wrong attempt returns one sentence of explanation.

3.4.3 Graphics and sound

Every picture is born from code: artgen.py draws the characters, trees, buildings and icons, and tiles.py builds the ground and the shapes of the world. world_map.py lays out eight regions, each with its own palette, weather and landmarks - coast, mangrove, karst towers, islands and the deep South - with rice terraces, paddies and a settlement in every town. The module audio.py plays six music tracks chosen to the mood of the scene, with a fallback that synthesises sound when the machine cannot play audio files.

3.4.4 Saving and settings

The save system stores the player's position, the pages collected, the Thai words learned, the culture cards and the playtime in one file, so the player can return to the same spot. The settings screen lets the player change keys, volume and display mode, and the choices are saved separately from the game file.

3.5 Testing

We wrote the test suite with Python's unittest library, one file per feature, so that the cause of a failure can be found quickly. The suite runs on a headless SDL driver, so it runs both on the development machine and on a teacher's laptop. Example test files are shown in Table 3.2; the whole suite of 22 files and 201 cases completes without a display in about 30 seconds.

Table 3.2 Example test files and what they check

Test file	Lines	What it checks
test_click.py	378	mouse response and click positions
test_world_map.py	330	sailing world, regions, landmarks and stop positions
test_travel.py	322	travel between stops and unlocking of the route
test_smoke.py	260	playing the Ko Tao quest end-to-end
test_landmarks.py	181	40 heritage sites: position, story and reward
test_cutscenes.py	108	the storm and the cup: shots, camera, hand-off

3.6 Publication and installation

The project is published as full source code with a requirements.txt file and a README explaining installation. Installation takes two commands - install the packages pygame-ce and numpy, then run main.py. The game works without an Internet connection after installation and needs no other program. Inside the game there is a settings screen and a key-help screen, so a new player can start without opening a document.

3.7 Equipment

Table 3.3 Equipment and software used in the project

Item	Specification
Computer	Ordinary classroom laptop (Intel i5-class CPU, 8 GB RAM, integrated graphics)
Language	Python 3.14.8 (CPython), compatible with 3.10+
Game library	pygame-ce 2.5.8 (SDL 2.32.10), the only graphical dependency
Numerical library	numpy 2.5.2 (used by the procedural art generator)
Testing	Python standard library unittest (no extra packages needed)
Editor / versions	Visual Studio Code; Git for version control; every feature on its own branch, then merged
Fonts / audio	TH Sarabun PSK by SIPA (SIL OFL); music tracks from credited free sources with an in-game credits file
Target display	Any 1280 x 720-or-larger desktop; the game draws at 640 x 320 internal pixels and scales 4x with nearest-neighbour (2560 x 1280), shrinking to fit the screen

Chapter 4 Results and Discussion

4.1 What was built

The delivered program is a complete, playable game: 17,762 lines of game code in 52 modules, plus a test suite of 3,283 lines - 75 files and 21,396 lines in all (Table 4.1). The interface is original pixel art generated by code at run time (artgen.py, 1,292 lines, plus tiles.py), so the project ships without any external image asset. The world carries more than 21,000 words of story, dialogue and lore: 24 stops in five acts, 40 landmarks, 25 side quests, 18 lore documents, 74 road encounters and two cutscenes. The longest module is the overworld (overworld.py, 2,126 lines), because it carries movement, quests, weather and the tablet; the story data is the second (story.py, 1,980 lines), because we deliberately keep knowledge in data files that other classes can edit.

Table 4.1 Main modules of the delivered game

Module	Lines	What it does
overworld.py	2,126	the walking world, quests, weather, tablet
story.py	1,980	24 stops: dialogue, words, cards, 24 puzzle specs
world_map.py	1,354	eight regions, towns, terraces and paddies
artgen.py	1,292	generates every pixel sprite in code
player_ui.py	861	HUD, journal (with the quest log), cards, inventory
tiles.py	816	tile maps and world geometry
landmarks.py	614	40 heritage sites with their stories
views.py	557	journal pages and page rewards

4.2 The teaching ladder: 24 puzzles, 8 archetypes

Every stop ends at a rune tablet. The tablet runs the player's code and checks it against the puzzle's spec. Eight archetypes let us test different shapes of Python knowledge while the story only grows in difficulty (Table 4.2). No lesson can be skipped: the pre-filled code in every tablet fails its own validator (Section 4.3).

Table 4.2 The eight puzzle archetypes and the Python concepts they teach

Archetype	N	Python concept being taught	Example stop
say	4	print() and program output	Chumphon: print("hold fast")
lines	4	exact multi-line output order	Ko Samui: three prints
set	4	variables and assignment	Ko Tao: turtles = 4
table	6	if/else, for loops, f-strings, lists	Ranong: water == "clear"
func	1	def, parameters, return values	Ko Lipe: send(worry)
safe	1	try / except TypeError	Satun: mend(page)

class	1	classes, attributes, methods	Yala: class Stamp
gauntlet	3	everything combined at key tablets	Promthep / Phi Phi / the finale

Each puzzle spec carries its own status text (the rule, written in story voice), a hint, and a praise line, so failure answers with teaching, never with mockery. The concepts follow the term order of the national computing curriculum: output, variables, conditions, repetition, collections, functions, error handling and classes.

4.3 Correctness evidence from the test suite

The project contains 22 test files and 201 automated test cases (3,283 test lines, including an ASCII map previewer and a frame renderer used for regression screenshots). They cover the story data (every stop has a unique word, every puzzle spec is well-formed), the game-state machine, quest and spark flows, mouse and keyboard handling, the text panel, save/load, audio behaviour, the sailing world with its regions and landmarks, the cutscenes, and end-to-end replays of the Ko Tao quest and the finale. The whole suite runs headless in about 30 seconds on our classroom laptop (Python 3.14.8), and under a minute with a cold world cache.

We checked two properties directly. First, the 24 solutions we prepared: we ran each one through the game's own validator in a separate process with a hard timeout, and all 24 passed. Second, the pre-filled starter code in every tablet was run through the same validator, and all 24 prefills were rejected, so the story does not advance until the code is corrected. Both checks gave the result the design intended. The test suite is part of the delivered source, so a teacher or a future class can run the same checks after editing a story file.

For player safety, submitted code runs in a restricted namespace with a whitelist of builtins, a 6,000-character limit, captured stdout, and a wall-clock timeout; when the player's code raises an error, the validator returns a short message and a hint instead of a traceback.

4.4 Screenshots of the delivered program

All six screenshots below are captures of the delivered build at 1920 x 1080, so the judge sees the program as a student sees it when the game is running in full screen.



Figure 4.1 The cover screen: title, generated pixel art and the menu (capture of the delivered build, 1920 x 1080)

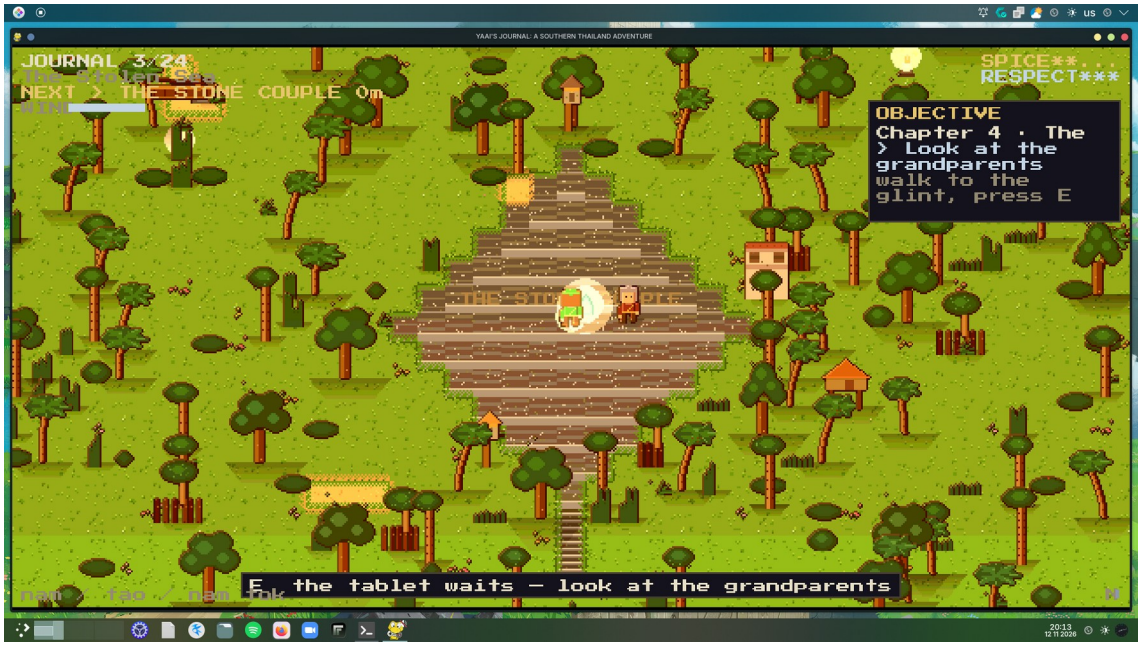


Figure 4.2 Walking the coast: the overworld with HUD, quest prompt and the stop beacon (full-screen capture)

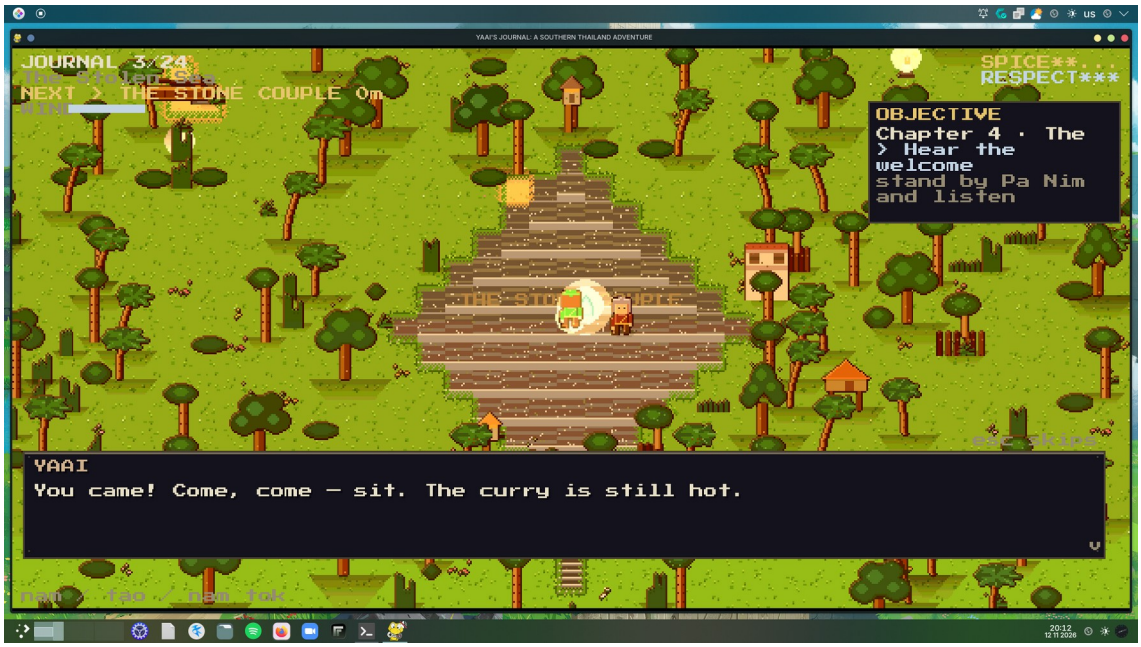


Figure 4.3 Typewriter dialogue: Noi teaches one beat of the story at a time (full-screen capture)



Figure 4.4 A rune tablet: the puzzle status rule and the prefilled code, ready to edit (full-screen capture)



Figure 4.5 The journal, opened from the overworld: collected pages, words and cards (full-screen capture)

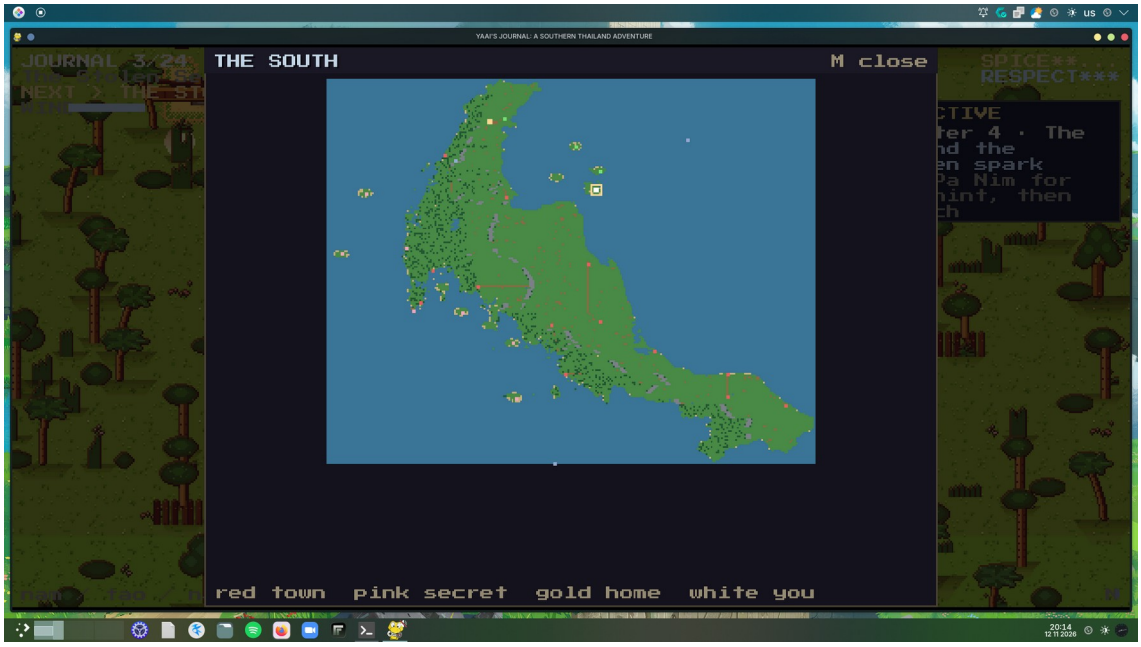


Figure 4.6 The sailing map: the 24 stops in five acts across the Southern coast

4.5 Discussion

Objective alignment. Each objective of Chapter 1 is answered by an artefact and, where the program itself can show it, a check: the teaching-aid objective by the game and its 24-puzzle ladder that the test suite runs through; the bilingual objective by the 24 Thai words with romanisation (Appendix A) learned through quests; the cultural objective by 24 place stories and 40 landmarks drawn from family accounts and local documents; the engineering

objective by the 201-case suite, the sandbox and the data-driven architecture; the documentation objective by this report and the classroom quiz in Appendix C.

Why real typed Python instead of blocks? Because Python is the language the syllabus examines. To keep beginners afloat we add scaffolds: the tablet shows the required output; the editor preloads commented, always-wrong code so editing is easier than writing from zero; and the first tablet is preceded by a 7-beat tutorial in which Noi teaches print, assignment and comparison. The validator answers every mistake with the ward's voice and the specific rule, which is exactly the immediate feedback flow requires.

Safety of player code. Player programs run in a restricted namespace: a whitelist of builtins (print, len, range, int, sorted, Exception handling, class building, and so on), captured stdout, a 6,000-character limit, and a wall-clock timeout. The runner is deliberately scoped to the classroom puzzles described in this report; it is a game, not a hardened sandbox, and the README explains this scope.

Performance and deployability. The game targets a 640 x 320 internal canvas scaled 4x (2560 x 1280, shrinking to fit the desktop) at 60 frames per second on integrated graphics; the whole test suite runs in about 30 s. Deploying in a classroom is: pip install pygame-ce numpy; python main.py. No installation and no Internet connection are needed after the two packages are cached.

Chapter 5 Conclusion and Discussion

5.1 Conclusions

The project met its objectives. We built an original educational game that teaches Python in the order the school curriculum requires, wrapped each lesson inside a factual story of the Southern coast, and ended with a working program: 24 puzzles whose answers are accepted, 24 starter programs that do not pass, a 201-case automated test suite, five acts that carry the culture of the coast, and a game that starts on an ordinary laptop from two pip packages.

Table 5.1 Objective-to-evidence summary

Objective / hypothesis	Result	Evidence
Teach Python through typed code	Met	8 archetypes, 24 puzzles; the validator verifies solutions (4.2).
24 Thai words inside the story	Met	24/24 words with romanisation + meaning (Appendix A).
Preserve Southern culture factually	Met	24 place stories, culture cards and 40 heritage landmarks drawn from local accounts.
Sandbox safety	Met	whitelist of builtins, captured stdout, timeout (4.5).
Maintainability	Met	Data-driven: a new province is a new dictionary; the 201-case suite guards the logic.
H1: lessons cannot be skipped	Checked	24 solutions accepted; 24 prefills rejected (4.3).
H2: content editable without engine changes	Checked	Stops are data dictionaries; the suite checks the data.

5.2 Discussion of the results

The design in which the story advances only when the code is correct proved workable: the player sees the next page only after passing the tablet, because the starter code on every tablet is written to fail. Hints and in-character explanations at every mistake make being stuck part of the story instead of the end of play.

A second lesson is that separating content from code affects the project's sustainability: adding a new stop is adding one dictionary, and the existing tests catch malformed data at once. A teacher who does not program can therefore join in editing the content, with the test suite as the safety net. We also learned that writing a puzzle's rule as one short

sentence helps players understand quickly, and keeping the validator's messages in the data file lets us tune the wording without touching the engine.

Because the game keeps no record of the player, the classroom quiz in Appendix C is what a teacher uses to see whether the lesson stayed. It takes ten minutes on paper, before and after a fifteen-minute session, and the difference between the two scores shows what the class gained. Questions 1 to 9 follow the teaching arc of the game itself, and question 10 checks the Thai word learned at Ko Tao, so the quiz also tells the teacher which stop to revisit with the class.

5.3 Problems encountered and how they were solved

- Wrong answers had to teach, not crash. Player code runs inside a restricted namespace, and every validator returns a short message and a hint instead of a traceback, so a mistake becomes the next lesson.
- The game had to run on an ordinary school laptop. All art is generated in code, the game needs only two pip packages, and it runs offline after they are cached.
- The tests had to run without a screen. The suite uses headless SDL drivers, so the 201 cases complete in about 30 seconds and can be run before every submission.
- Content had to stay editable by other classes. Stops, words, cards and puzzle specs live in data dictionaries, so a new province is a new dictionary, not a rewrite.
- No lesson may be skipped. Every pre-filled starter program fails its own validator; the 24/24 rejection check is part of the delivered evidence.

5.4 Suggestions and future development

- Content packs: teacher-editable stop dictionaries for other provinces (Central, Isan, North) and other subjects (mathematics, science) using the same engine and validators.
- A session summary a teacher can print: which stops the class reached, how many attempts each took and how often hints were used, kept on the classroom computer only.
- A Thai/English UI toggle and screen-reader-friendly font scaling.
- A Linux/macOS CI runner in the school lab, so every future content pack is checked automatically at submit time.

We close where the game closes: at the Wai Khru. We thank the teachers we had, and we hope this program becomes the teaching aid a future class of ours will borrow.

References

[1] Beck, K. (2003). Test-driven development: by example. Addison-Wesley.

[2] Csikszentmihalyi, M. (1990). Flow: The psychology of optimal experience. Harper & Row.

[3] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining "gamification". Proceedings of MindTrek 2011, 9-15.

[4] Institute for the Promotion of Teaching Science and Technology (IPST). (2017). Standards and indicators of the computing-science curriculum, Basic Education (revised). IPST. (In Thai.)

[5] MacLeod, K. (2005-2019). Music tracks used in the game [Audio files, CC BY 4.0]. Kevin MacLeod / incompetech. Credited in the project's CREDITS.md.

[6] Papert, S. (1980). Mindstorms: Children, computers, and powerful ideas. Basic Books.

[7] Prensky, M. (2001). Digital game-based learning. McGraw-Hill.

[8] Python Software Foundation. (2026). The Python tutorial (version 3.14) [Documentation]. https://docs.python.org/3/tutorial/

[9] pygame-ce community. (2026). pygame-ce documentation (version 2.5.8) [Documentation]. https://pyga.me/

[10] Software Industry Promotion Agency (SIPA). (2010). TH Sarabun PSK [Font, SIL Open Font License]. Source repository copy bundled with game assets (CREDITS file included).

[11] Sobel, D. (2004). Place-based education: Connecting classrooms and communities. Orion Society.

[12] Vygotsky, L. S. (1978). Mind in society: The development of higher psychological processes. Harvard University Press.

[13] Weintrop, D., & Wilensky, U. (2017). To block or not to block, that is the question: Students' perceptions of blocks-based programming. ACM Transactions on Computing Education, 17(3), 1-25.

Appendix A The 24 stops, their Thai words and their puzzles

#	Stop and place title	Thai word	Reads as	Means	Puzzle (concept)
1	Chumphon: The Signal Hill	น้ำ	nam	water	say (print())
2	Ko Tao: The Turtle Watch	เต่า	tao	turtle	set (variables)
3	Ko Pha-ngan: The King's Initials	น้ำตก	nam tok	waterfall	say (print())
4	Ko Samui: The Stone Couple	หิน	hin	stone	lines (exact output order)
5	Surat Thani - Chaiya: The Rope Pull	แรง	raeng	strength	gauntlet (combination)
6	Ranong: The Tin Pan	แร่	rae	ore	table (if / for / f-strings)
7	Surin Islands: Reading the Sky	ทะเล	tha le	sea	table (if / for / f-strings)
8	Khao Lak: The Lantern	รำลึก	ram luek	remember	say (print())
9	Similan Islands: Nine Islands	เก้า	kao	nine	lines (exact output order)
10	Phang Nga: The Floating School	เกาะ	ko	island	table (if / for / f-strings)
11	Phuket Old Town: The Beacon Chain	หญิง	ying	woman	table (if / for / f-strings)
12	Promthep Cape: The Golden Hour	ดวงอาทิตย์	duang atit	sun	table (if / for / f-strings)
13	Ko Phi Phi: The Steady Lamp	ถ้ำ	tham	cave	lines (exact output order)
14	Krabi: Twelve Hundred Steps	แม่	mae	mother	gauntlet (combination)
15	Nakhon Si Thammarat: The Dancing Hands	ครู	khru	teacher	set (variables)
16	Phatthalung: The Little Brother	หนัง	nang	leather	set (variables)
17	Trang: Rubber Dawn	ยาง	yang	rubber	lines (exact output order)
18	Ko Kradan: The Gentle Hour	ปะการัง	pa ka rang	coral	say (print())
19	Satun - Pak Bara: The Torn Logbook	เรือ	ruea	boat	safe (try / except)
20	Ko Lipe: Letting Go	เรือไฟ	ruea fai	floating boat	func (def / return)
21	Songkhla: The Loom	ท่าเรือ	tha ruea	harbour	set (variables)
22	Pattani: The Painted Bow	มัสยิด	masjid	mosque	table (if / for / f-strings)

23	Yala: Wax and Colour	มลาย	ma lai	Malay	class (classes)
24	Narathiwat: The Last Chorus	ร้อง	rong	to sing	gauntlet (combination)

Table A.1 The complete puzzle inventory, generated directly from story.py of the delivered game (24 of 24).

Appendix B Three example tablets and their solutions

The three solutions below are the ground floor, the middle and the top of the ladder: a variable at Ko Tao, a loop plus output at Surat Thani, and a class at Yala. The validator accepts exactly these shapes (plus equivalent correct answers).

B.1 Ko Tao: variables (archetype: set)

```
turtles = 4
```

B.2 Surat Thani (Chaiya): loop (archetype: table)

```
rope = 4
for i in range(rope):
    print("pull")
```

B.3 Yala: class (archetype: class)

```
class Stamp:
    color = "blue"
    def press(self, cloth):
        return cloth + " batik"
```

Appendix C How to run, the project tree, the test suite and the classroom quiz

C.1 How to run (no installation needed beyond two packages)

```
python -m pip install -r requirements.txt   # pygame-ce, numpy
python main.py
```

Runs on Windows 10/11, Linux or macOS with Python 3.10+; the game opens a 2560 x 1280 window at 4x the 640 x 320 canvas (shrinking to fit the desktop), remembers options and saves in the user's config folder.

C.2 Project tree (abridged)

```
new-game-project/
  main.py                  entry point and game loop
  requirements.txt         pygame-ce>=2.5, numpy>=2.0
  README.md                how to play and how it is built
  game/                    52 modules (37 game files + 15 scenes)
    story.py               the whole curriculum: 24 stops + 24 puzzles
    tablets.py  runner.py   validators + safe player-code sandbox
    artgen.py   tiles.py    code-drawn art and terrain
    scenes/                overworld, stop, cutscene, hub, finale ...
  tests/                   22 files, 201 cases, headless
    tools/                 ascii preview + frame renderer
```

C.3 The automated test suite

Test file	Lines	What it guards
test_click.py	378	mouse handling incl. click-to-run on the tablet
test_world_map.py	330	sailing map, regions, landmarks, unlock order
test_travel.py	322	travel between stops, sparks, quest order
test_smoke.py	260	end-to-end: boot, cover, overworld, Ko Tao quest, finale
test_landmarks.py	181	40 heritage sites: placement, story, reward
test_content_expansion.py	172	25 side quests, 74 encounters, host lines

test_game_state.py	167	quests, pages, words, cards, journey state machine
test_mobs.py	166	NPC behaviour, fish and companions
test_plot.py	142	prologue/finale script integrity
test_story.py	141	every stop: unique word, well-formed puzzle spec
test_cutscenes.py	108	the storm and the cup: shots, camera, hand-off
test_tablets.py + others	77+	validators, save/load, audio, keymap, icons (8 files)

Table C.1 The test inventory (22 test files, 201 cases, 3,283 lines; run headless in about 30 s).

C.4 Classroom quiz and answer key

One session of 25 minutes: ten for the quiz, fifteen for play. Give the same ten questions on paper before and after, closed book. One point per question; the difference between the two scores belongs to that class alone, so we publish no figures of our own.

No.	Question	Choices	Key
1	Which line shows the word hold fast on the screen?	A) hold fast = "print" B) print("hold fast") C) print(hold fast)	B
2	After turtles = 4, what is turtles?	A) text B) the number 4 C) nothing yet	B
3	Which operator asks a question (is it equal)?	A) = B) == C) =>	B
4	How many times does this print pull? for i in range(4): print("pull")	A) 3 B) 4 C) 5	B
5	water = "clear"; if water == "clear": print("keep") else: print("swirl") - what prints?	A) keep B) swirl C) nothing	A
6	lamp = 0; while lamp < 3: print("steady"); lamp = lamp + 1 - how many steady lines?	A) 2 B) 3 C) forever	B
7	What does send("rain") return here? def send(worry): return "gone: " + worry	A) rain B) gone: rain C) an error	B
8	Which block catches a mistake without crashing the program?	A) if ... else B) for ... in C) try ... except	C
9	Stamp().press("silk") returns what, given class Stamp: def press(self, cloth): return cloth + " batik"	A) silk batik B) blue C) batik silk	A
10	The word เต่า (tao), earned at Ko Tao, means ...	A) turtle B) island C) wave	A

Table C.2 The classroom quiz. Questions 1-9 follow the game's own teaching arc; question 10 checks the Thai vocabulary of the stops.